DATABASE MANAGEMENT SYSTEM FOR HOSPITAL

Çağrı Çaycı

June 2024

1 Introduction

In this project, a database management system for a hospital is developed to streamline hospital management and facilitate easier access to necessary information. A comprehensive database schema is designed, data is uploaded, and the specified operations from the homework definition are implemented.

1.1 Database Schema

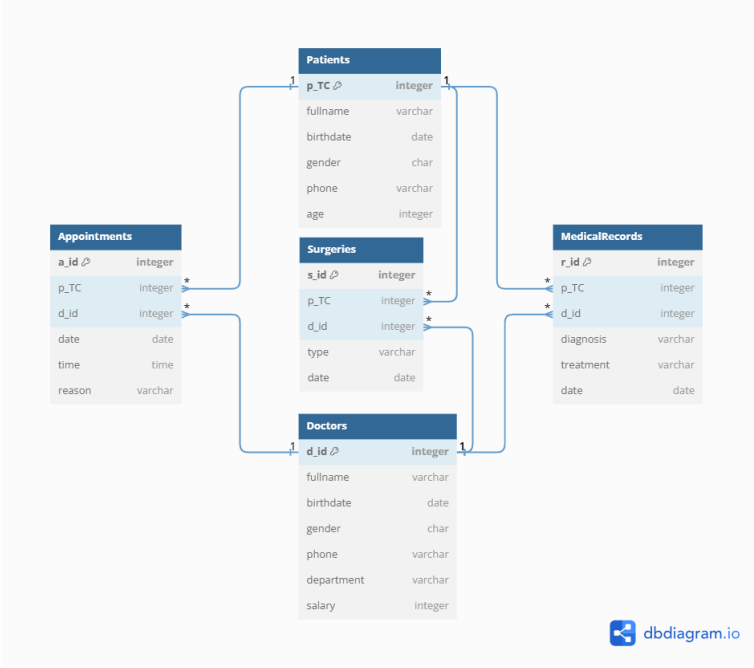


Figure 1: Schema of Database

Figure 1 represents a relational database schema for a healthcare system. It consists of five tables: **Patients**, **Doctors**, **Appointments**, **Surgeries**, and **MedicalRecords**. Below is a description of each table and the relationships between them:

- **Tables**:
  - **Patients:** This table stores essential information about patients, such as their TC (Turkish Citizenship) number, full name, birthdate, etc.
  - **Doctors:** It holds information about doctors, including their ID, full name, birthdate, etc.
  - **Appointments:** Here, details about appointments are recorded, including appointment ID, patient TC, doctor ID, date, time, and reason for the appointment.
  - **Surgeries:** Information about surgeries is stored in this table, including surgery ID, patient TC, doctor ID, type of surgery, and date of the surgery.
  - **MedicalRecords:** This table contains medical records with information such as record ID, patient TC, doctor ID, diagnosis, treatment, and the date of the record.
- **Relations**
  - **Patients to Appointments:** Each patient can have multiple appointments, but each appointment is linked to only one patient.
  - **Patients to Surgeries:** Similarly, each patient can undergo multiple surgeries, but each surgery is associated with only one patient.
  - **Patients to MedicalRecords:** Each patient can have multiple medical records, but each medical record pertains to only one patient.
  - **Doctors to Appointments:** A doctor can have multiple appointments, but each appointment is tied to only one doctor.
  - **Doctors to Surgeries:** Likewise, a doctor can perform multiple surgeries, but each surgery is performed by only one doctor.
  - **Doctors to MedicalRecords:** Each doctor can be associated with multiple medical records, but each medical record is linked to only one doctor.

2 Functions and Procedures

Functions and procedures with the features given in the homework have been created and shown in this section.

2.1 Get Medical Records of a Patient

The procedure in Figure 2 has been developed to retrieve medical records of a patient using their unique identifier, referred to as p_TC. Figure **??** displays the medical results of all patients. When the procedure is invoked with the p_TC value of 1001, it returns the medical records corresponding to the patient with p_TC equal to 1001, as illustrated in Figure 4.

```
DELIMITER //
DROP PROCEDURE IF EXISTS getMedicalRecords;
CREATE PROCEDURE getMedicalRecords(IN p_TC_in INTEGER)
BEGIN
    SELECT *
    FROM MedicalRecords
    WHERE p_TC_in = p_TC;
END; //
DELIMITER ;

CALL getMedicalRECORDS(1001);
```

Figure 2: The procedure to get medical records.

r_id	p_TC	d_id	diagnosis	treatment	date
4001	1001	2001	High blood pressure	Prescribed medication	2024-06-10
4002	1002	2002	Flu	Antiviral medication	2024-06-12
4003	1003	2003	Broken leg	Leg cast	2024-06-15
4004	1004	2004	Migraine	Painkillers	2024-06-18
4005	1005	2005	Stage 3 cancer	Chemotherapy	2024-06-20
4006	1006	2006	Contact dermatitis	Topical cream	2024-06-22
4007	1007	2007	Prostate enlargement	Prescribed medication	2024-06-25
4008	1008	2008	Polyp	Removed during colonoscopy	2024-06-28
4009	1009	2009	Ear infection	Antibiotics	2024-06-30
4010	1010	2010	Depression	Therapy sessions	2024-07-02
4011	1001	2002	Influenza	Antiviral medication	2024-07-05
4012	1001	2003	Fractured arm	Arm cast	2024-07-08
4013	1001	2004	Migraine	Painkillers	2024-07-10
NULL	NULL	NULL	NULL	NULL	NULL

Figure 3: All medical records

r_id	p_TC	d_id	diagnosis	treatment	date
4001	1001	2001	High blood pressure	Prescribed medication	2024-06-10
4011	1001	2002	Influenza	Antiviral medication	2024-07-05
4012	1001	2003	Fractured arm	Arm cast	2024-07-08
4013	1001	2004	Migraine	Painkillers	2024-07-10

Figure 4: Medical records of a patient

2.2 Number of Unique Patients Who Underwent Surgery

The procedure depicted in Figure 7 involves creating a temporary table for unique patients, where all surgeries are iterated. If a surgery is not already in the unique patients table, it is added. Finally, the procedure concludes by displaying the number of entries in the unique patients table. As evident from both Figure 6 and Figure 7, it operates correctly.

```
DELIMITER //
DROP PROCEDURE IF EXISTS numberOfUniquePatientsSurgery;
CREATE PROCEDURE numberOfUniquePatientsSurgery()
BEGIN
    DECLARE counter INTEGER DEFAULT 0;
    DECLARE number_of_surgeries INTEGER DEFAULT 0;
    DECLARE number_of_uniquePatients INTEGER DEFAULT 0;
    DECLARE p_TC VARCHAR(50);
    DECLARE fullname VARCHAR(100);
    DECLARE birthdate DATE;
    DECLARE gender CHAR(1);
    DECLARE phone VARCHAR(20);

    CREATE TEMPORARY TABLE IF NOT EXISTS uniquePatients(
        TC VARCHAR(50),
        fullname VARCHAR(100),
        birthdate DATE,
        gender CHAR(1),
        phone VARCHAR(20)
    );

    SELECT COUNT(*) INTO number_of_surgeries FROM Surgeries;

    WHILE counter < number_of_surgeries DO
        SELECT p_TC, fullname, birthdate, gender, phone
        INTO p_TC, fullname, birthdate, gender, phone
        FROM Surgeries
        LIMIT counter, 1;

        IF NOT EXISTS (SELECT 1 FROM uniquePatients WHERE TC = p_TC) THEN
            INSERT INTO uniquePatients (TC, fullname, birthdate, gender, phone)
            VALUES (p_TC, fullname, birthdate, gender, phone);
        END IF;

        SET counter = counter + 1;
    END WHILE;

    SELECT COUNT(*) INTO number_of_uniquePatients FROM uniquePatients;

    SELECT number_of_uniquePatients AS NumberOfUniquePatients;
END; //
DELIMITER ;

CALL numberOfUniquePatientsSurgery;
```

Figure 5: The procedure for number of unique patients

	s_id	p_TC	d_id	type	date
▶	5001	1003	2003	Leg fracture repair	2024-06-16
	5002	1005	2005	Tumor removal	2024-06-21
	5003	1006	2006	Dermabrasion	2024-06-23
	5004	1007	2007	Prostatectomy	2024-06-26
	5008	1008	2008	Colon polypectomy	2024-06-29
	5009	1009	2009	Tympanoplasty	2024-07-01
	5010	1010	2010	None	2024-08-15
	5011	1001	2003	Leg fracture repair	2024-08-16
	5012	1001	2005	Tumor removal	2024-05-23
	5013	1001	2006	Dermabrasion	2024-04-15
*	NULL	NULL	NULL	NULL	NULL

Figure 6: All surgeries

	NumberOfUniquePatients
▶	10

Figure 7: Number of unique patient who underwent surgery

2.3 Rate of Surgery over Medical Records

The function in Figure 8 has been developed to determine the ratio of the number of surgeries a patient has undergone to the number of health records the patient has. It counts the number of health records with a given p_{TC} and the number of surgeries with a given p_{TC}, and returns the rate between them.

```
DROP FUNCTION IF EXISTS patientSurgeryAppointmentRate;
DELIMITER //
CREATE FUNCTION patientSurgeryAppointmentRate(p_TC INTEGER)
RETURNS DECIMAL(10,2)
READS SQL DATA
DETERMINISTIC
BEGIN
    DECLARE number_of_medical_records INTEGER DEFAULT 0;
    DECLARE number_of_surgeries INTEGER DEFAULT 0;
    DECLARE surgeryAppointmentRate DECIMAL(10,2);

    SELECT COUNT(*) INTO number_of_medical_records FROM MedicalRecords WHERE MedicalRecords.p_TC = p_TC;
    SELECT COUNT(*) INTO number_of_surgeries FROM surgeries WHERE surgeries.p_TC = p_TC;

    IF number_of_medical_records > 0 THEN
        SET surgeryAppointmentRate = number_of_surgeries / number_of_medical_records;
    ELSE
        SET surgeryAppointmentRate = 0;
    END IF;

    RETURN surgeryAppointmentRate;
END; //
DELIMITER ;

SELECT p_TC, patientSurgeryAppointmentRate(p_TC) FROM patients;
```

Figure 8: The function to get surgery/appointment rate

a_id	p_TC	d_id	date	time	reason
3001	1001	2001	2024-06-10	09:00:00	Routine checkup
3002	1002	2002	2024-06-12	10:30:00	Vaccination
3003	1003	2003	2024-06-15	11:15:00	Fracture treatment
3004	1004	2004	2024-06-18	14:00:00	Headache consultation
3005	1005	2005	2024-06-20	15:30:00	Chemotherapy session
3006	1006	2006	2024-06-22	16:45:00	Skin rash examination
3007	1007	2007	2024-06-25	09:30:00	Prostate checkup
3008	1008	2008	2024-06-28	13:00:00	Colonoscopy
3009	1009	2009	2024-06-30	11:45:00	Ear infection treatment
3010	1010	2010	2024-07-02	10:00:00	Therapy session
NULL	NULL	NULL	NULL	NULL	NULL

Figure 9: All Appointments

p_TC	patientSurgeryAppointmentRate(p_TC)
1001	0.75
1002	0.00
1003	1.00
1004	0.00
1005	1.00
1006	1.00
1007	1.00
1008	1.00
1009	1.00
1010	1.00

Figure 10: Rate of surgery/medical records

3 Triggers

Triggers with the features given in the homework have been created and shown in this section.

3.1 Postponing Appointments

A 'before insert trigger' is implemented as depicted in Figure 11 to address potential issues associated with scheduling patient appointments earlier than usual, which could be problematic for patients. This trigger exclusively permits postponing the appointment date to a later time than the default. Any attempt to schedule an appointment earlier than usual will result in an error, preventing the update from being executed as shown in Figure 12.

```
DROP TRIGGER IF EXISTS postponeTrigger;
DELIMITER //
CREATE TRIGGER postponeTrigger
BEFORE UPDATE ON Appointments
FOR EACH ROW
BEGIN
    IF old.date > new.date THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot update appointment to a date in the past';
    END IF;
END; //
DELIMITER ;

UPDATE Appointments SET date = date + INTERVAL 3 DAY WHERE a_id = 3001;
UPDATE Appointments SET date = date - INTERVAL 3 DAY WHERE a_id = 3001;
```

Figure 11: The trigger to postpone appointment

```
UPDATE Appointments SET date = date + INTERVAL 3 DAY WHERE a_id = 3001  1 row(s) affected Rows matched: 1  Changed: 1  Warnings: 0
UPDATE Appointments SET date = date - INTERVAL 3 DAY WHERE a_id = 3001  Error Code: 1644. Cannot update appointment to a date in the past
```

Figure 12: Postpone trigger behaviours

3.2 Setting Follow-up Appointments After Surgery

An 'after insert trigger' is implemented, as depicted in Figure 13, to schedule an appointment for the patient 15 days after the surgery. This delay is necessary to ensure the patient's health situation is considered. As illustrated in Figure 14, even though there is no appointment scheduled for the patient with p_{TC} 1002 on 2024-10-17, as shown in Figure 9, one is created because there is a surgery scheduled for 2024-10-02.

```sql
DROP TRIGGER IF EXISTS insertAppointmentAfterSurgery;
DELIMITER //
CREATE TRIGGER insertAppointmentAfterSurgery
AFTER INSERT ON Surgeries
FOR EACH ROW
BEGIN
    DECLARE last_id INT;

    SELECT MAX(a_id) INTO last_id FROM Appointments;

    IF last_id IS NULL THEN
        SET last_id = 0;
    END IF;

    INSERT INTO Appointments (a_id, p_TC, d_id, date, time, reason)
    VALUES (last_id + 1, NEW.p_TC, NEW.d_id, NEW.date + INTERVAL 15 DAY, '09:00:00', 'Surgery Control');
END; //
DELIMITER ;

INSERT INTO Surgeries VALUES(5014, 1002, 2010, 'Leg Operation',  '2024-10-02');
Select Surgeries.p_TC, Surgeries.d_id, Surgeries.date as SurgeryDate, Appointments.date as AppointmentDate FROM Surgeries, Appointments
    WHERE Surgeries.p_TC = Appointments.p_TC and Surgeries.p_TC = 1002 and Surgeries.d_id = Appointments.d_id;
```

Figure 13: The trigger to insert appointment

	p_TC	d_id	SurgeryDate	AppointmentDate
▶	1002	2010	2024-10-02	2024-10-17

Figure 14: Appointment after surgery triggered

3.3 Removing Appointments for Some Surgery

When scheduling surgery appointments for patients over the age of 75, it is anticipated that the surgery may take longer due to potential complications associated with their age. To preemptively address any issues, an 'insert before trigger' was implemented, as depicted in Figure 15. This trigger cancels all appointments for that doctor on the specified day, ensuring adequate time is allocated for the surgery. As illustrated in Figure 16, the appointment on 2024-06-15 is canceled, contrary to what is shown in Figure 9, due to the surgery of an elderly patient.

```sql
DROP TRIGGER IF EXISTS insertSurgery;
DELIMITER //
CREATE TRIGGER insertSurgery
BEFORE INSERT ON Surgeries
FOR EACH ROW
BEGIN
    DECLARE patient_age INT DEFAULT 0;
    SELECT age INTO patient_age from patients where patients.p_TC = new.p_TC;
    IF patient_age > 75 THEN
        DELETE FROM Appointments WHERE Appointments.date = new.date and new.p_TC != Appointments.p_TC;
    END IF;
END; //
DELIMITER ;


INSERT INTO Surgeries VALUES(5015, 1010, 2002, 'Brain Surgery', '2024-06-15');
SELECT * FROM Appointments;
```

Figure 15: The trigger to insert surgery

	a_id	p_TC	d_id	date	time	reason
▶	3001	1001	2001	2024-06-13	09:00:00	Routine checkup
	3002	1002	2002	2024-06-12	10:30:00	Vaccination
	3004	1004	2004	2024-06-18	14:00:00	Headache consultation
	3005	1005	2005	2024-06-20	15:30:00	Chemotherapy session
	3006	1006	2006	2024-06-22	16:45:00	Skin rash examination
	3007	1007	2007	2024-06-25	09:30:00	Prostate checkup
	3008	1008	2008	2024-06-28	13:00:00	Colonoscopy
	3009	1009	2009	2024-06-30	11:45:00	Ear infection treatment
	3010	1010	2010	2024-07-02	10:00:00	Therapy session
	3011	1002	2010	2024-10-17	09:00:00	Surgery Control
	3012	1010	2002	2024-06-30	09:00:00	Surgery Control
*	NULL	NULL	NULL	NULL	NULL	NULL

Figure 16: Insert surgery triggered

3.4 Showing Triggers

Figure 17 illustrates all active triggers in the database. Each trigger is designed to take action based on its defined conditions. The following code is used to show all triggers:

```
SHOW TRIGGERS;
```

	Trigger	Event	Table	Statement	Timing	Created
▶	postponeTrigger	UPDATE	appointments	BEGIN IF old.date > new.date THEN SI...	BEFORE	2024-06-04 18:21:39.97
	insertSurgery	INSERT	surgeries	BEGIN DECLARE patient_age INT DEFAULT 0...	BEFORE	2024-06-04 19:26:41.70
	insertAppointmentAfterSurgery	INSERT	surgeries	BEGIN DECLARE last_id INT; -- Get the l...	AFTER	2024-06-04 18:35:52.43

Figure 17: All triggers

4 Transactions

Transactions with the features given in the homework have been created and shown in this section.

4.1 Giving a Raise To The Best Doctor

Similar to the "Employee of the Month" program, the hospital has implemented a transaction, as shown in Figure 18, where the salary of the doctor with the most appointments has been doubled. Despite the salary of doctors shown in Figure 19, the salary of the most hardworking doctor, with ID 2010, has been increased twofold.

```
DELIMITER //
START TRANSACTION;

SET @best_doctor_id := NULL;

SELECT d_id INTO @best_doctor_id
FROM (
    SELECT d_id, COUNT(*) as count
    FROM appointments
    GROUP BY d_id
    ORDER BY count DESC
    LIMIT 1
) AS counts;

UPDATE doctors
SET salary = CASE WHEN @best_doctor_id IS NOT NULL THEN salary * 2 ELSE salary END
WHERE d_id = @best_doctor_id;
COMMIT; //

DELIMITER ;

SELECT * FROM Doctors;
```

Figure 18: The transaction for raising salary of best doctor

d_id	fullname	birthdate	gender	phone	department	salary
2001	Dr. Michael Johnson	1970-09-05	M	111-111-1111	Cardiology	120000
2002	Dr. Emily White	1982-03-15	F	222-222-2222	Pediatrics	110000
2003	Dr. David Brown	1975-11-20	M	333-333-3333	Orthopedics	125000
2004	Dr. Sarah Miller	1988-07-10	F	444-444-4444	Neurology	130000
2005	Dr. James Lee	1972-01-30	M	555-555-5555	Oncology	135000
2006	Dr. Anna Clark	1980-05-18	F	666-666-6666	Dermatology	115000
2007	Dr. William Martinez	1990-12-25	M	777-777-7777	Urology	125000
2008	Dr. Samantha Taylor	1985-08-08	F	888-888-8888	Gastroenterology	130000
2009	Dr. Daniel Hernandez	1977-04-01	M	999-999-9999	ENT	120000
2010	Dr. Jessica Adams	1993-06-10	F	000-000-0000	Psychiatry	110000
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 19: All doctors

d_id	fullname	birthdate	gender	phone	department	salary
2001	Dr. Michael Johnson	1970-09-05	M	111-111-1111	Cardiology	120000
2002	Dr. Emily White	1982-03-15	F	222-222-2222	Pediatrics	110000
2003	Dr. David Brown	1975-11-20	M	333-333-3333	Orthopedics	125000
2004	Dr. Sarah Miller	1988-07-10	F	444-444-4444	Neurology	130000
2005	Dr. James Lee	1972-01-30	M	555-555-5555	Oncology	135000
2006	Dr. Anna Clark	1980-05-18	F	666-666-6666	Dermatology	115000
2007	Dr. William Martinez	1990-12-25	M	777-777-7777	Urology	125000
2008	Dr. Samantha Taylor	1985-08-08	F	888-888-8888	Gastroenterology	130000
2009	Dr. Daniel Hernandez	1977-04-01	M	999-999-9999	ENT	120000
2010	Dr. Jessica Adams	1993-06-10	F	000-000-0000	Psychiatry	220000
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 20: Doctors' new salaries

4.2 Updating Medical Records

In the transaction shown in Figure 21, updating medical records after an appointment is simulated. Despite the previous medical records shown in Figure 16, a new medical record is added to the medical records as shown in Figure 22.

```
DELIMITER //
START TRANSACTION;
SET @last_id := NULL;
SELECT MAX(a_id) INTO @last_id FROM Appointments;
INSERT INTO Appointments (a_id, p_TC, d_id, date, time, reason)
VALUES (@last_id + 1, 1003, 2004, '2024-06-10', '12:00:00', 'Follow-up Check-up');

UPDATE MedicalRecords
SET diagnosis = 'Improved Arthritis Condition', treatment = 'Continued Physical Therapy'
WHERE p_TC = 1003;


COMMIT; //

DELIMITER ;

SELECT * FROM MedicalRecords;
```

Figure 21: The transaction for Updating medical records after appointment

r_id	p_TC	d_id	diagnosis	treatment	date
4001	1001	2001	High blood pressure	Prescribed medication	2024-06-10
4002	1002	2002	Flu	Antiviral medication	2024-06-12
4003	1003	2003	Improved Arthritis Condition	Continued Physical Therapy	2024-06-15
4004	1004	2004	Migraine	Painkillers	2024-06-18
4005	1005	2005	Stage 3 cancer	Chemotherapy	2024-06-20
4006	1006	2006	Contact dermatitis	Topical cream	2024-06-22
4007	1007	2007	Prostate enlargement	Prescribed medication	2024-06-25
4008	1008	2008	Polyp	Removed during colonoscopy	2024-06-28
4009	1009	2009	Ear infection	Antibiotics	2024-06-30
4010	1010	2010	Depression	Therapy sessions	2024-07-02
4011	1001	2002	Influenza	Antiviral medication	2024-07-05
4012	1001	2003	Fractured arm	Arm cast	2024-07-08
4013	1001	2004	Migraine	Painkillers	2024-07-10
NULL	NULL	NULL	NULL	NULL	NULL

Figure 22: New medical records

5 Conclusion

In conclusion, this project has effectively established a database schema for a hospital management system and implemented several operations, accompanied by illustrative examples to demonstrate their functionality in accordance with the homework requirements. As a result, this work lays a strong groundwork for the development of a more sophisticated system with comprehensive functionality.

6 Development Environment

This project was developed using MySQL Workbench on Windows.